Robotic Manipulation

Perception, Planning, and Control

Russ Tedrake

© Russ Tedrake, 2020-2026
Last modified 2026-10-11.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Fall 2026 semester.

Previous Chapter Table of contents Next Chapter

Chapter 7
Mobile Manipulation

So far we've focused primarily on table-top manipulation. You can bring many interesting objects / challenges to your robot that is fixed to the table, and this will continue to keep us busy for many chapters to come. But before we get too much further, I'd like to stop and ask -- what happens if the robot also has wheels (or legs!)?

Many of the technical challenges in "mobile manipulation" are the same challenges we've been discussing for table-top manipulation. But there some challenges that are new, or at least are more urgent in the mobile setting. Describing those challenges, and some solutions, is my primary goal for this chapter.

Most importantly, adding mobility to our manipulators can often help us increase the scope of our ambitions for our robots. Now we can think about robots moving through houses and accomplishing diverse tasks. It's time to send our creations out to discover (manipulate?) the world!

7.1 A New Cast of Characters

7.2 What's different about perception?

7.2.1 Partial views / active perception

We discussed some of the technical challenges that come with having only partial views of the objects that you wish to manipulate in the geometric perception chapter. In the table-top manipulation setting, we can typically put cameras looking onto the table from all external angles, but our views can still be occluded in clutter.

When you're a mobile robot, we typically don't assume that you can fully instrument your environment to the same extent. Typically we try to work with only robot-mounted sensors, such as head-mounted cameras. Our antipodal grasping strategy won't work out of the box if we only see one side of each object.

The primary solution to this problem is to use machine learning. If you only see one half of a mustard bottle, then the instantaneous sensor readings simply don't contain enough information to plan a grasp; somehow we must infer something about what the back of the object looks like (aka "shape completion", though it need not be done explicitly). The reason that you would have a good guess at what the back half of the mustard bottle looks like is because you've interacted with mustard bottles before -- it is the statistics of the objects you've interacted with that provides the missing information. So learning isn't just a convenience here, it's fundamental. We'll discuss learning-based perception soon!

There is another mitigation, though, that is often available to a mobile manipulator. Now that the cameras (and all of the sensors) are mobile, we can actively move them in order to gather information / reduce uncertainty. There is a rich literature for "active perception" and "planning under uncertainty". [More to come]

7.2.2 Unknown (potentially dynamic) environments

For table-top manipulation, we started by assuming known objects and then discussed approaches (such as antipodal grasping) that could work on unknown objects. But throughout that discussion, we always assumed that we knew the geometry and location of the table / bins! For instance, we actively cropped away the point cloud returns that hit the known environment and left only points associated with the objects. That's a very reasonable thing to do when your robot is bolted to the table, but we need to relax that assumption once we go mobile, and start interacting with more diverse / less known environments.

One *could* try to model everything in the environment. We could do object detection to recognize the table, and pose estimation (possibly shape estimation) to fit a parametric table model to the observations. But that's a difficult road to travel, especially in diverse and expansive scenes. Is there an analogy to the antipodal grasping for objects that can similarly reduce our need for explicit modeling of the environment?

We'd like to have a representation of the environment that does not require explicit object models, that supports the types of queries we need for motion planning (for instance, fast collision detection and minimum-distance computations), which we can update efficiently from our raw sensor data, and which scales well to large scenes. Raw, merged point clouds, for instance, are easy to update and don't require a model, but would not be ideal for fast planning queries.

Voxel grids are a geometry representation that meets (almost all of) our desiderata -- we actually used them already in order to efficiently down-sample a point cloud. In this representation we discretize some finite volume of 3D space into a grid of fixed-size cubes; let's say 1 cm^3. We mark each of these cubes as either occupied or unoccupied (or more generally, we can have a probability of being occupied). Then, for collision-free motion planning, we treat the occupied cubes as collision geometry to be avoided. Many collision queries with voxel grids can be fast (and are easily parallelized). In particular the sphere-on-voxel collision query can be very fast, which is one of the reasons that you see a number of collision geometries in the Drake models that are approximated with densely packed spheres. Updating a voxel grid with points clouds is also efficient. In order to scale voxel representations to have both fine resolution and to be able to represent very large scenes, we can make use of clever data structures.

Example 7.1 (Octomap)

One particularly nice and efficient implementation of voxel grids at scale for robotics is an algorithm / software package called Octomap [1]...

Segmenting the visual scene into objects vs environment these days is best done with machine learning; we'll implement segmentation pipelines soon.

7.2.3 Robot state estimation

The biggest difference that comes up in perception for a mobile manipulator is the need to estimate not only the state of the environment, but the state of the robot. For instance, in our point cloud processing algorithms so far, we have tacitly assumed that we know ${}^{W}X^{C}$, the pose of the camera in the world. We could potentially invest in elaborote calibration procedures, using known shapes and patterns, in order to estimate (offline) these camera extrinsics. We've also take the kinematics of the robot to be known -- this is a pretty reasonable assumption when the robot is bolted to the table and all of the transforms between the world and the end-effector are measured directly with high-accuracy encoders. But these assumptions break down once the robot has a mobile base.

State estimation for mobile robots has a strong history; [2] is the canonical reference. Often these algorithms are based on (approximate) recursive Bayesian filtering, or the closely related smoothing algorithms [3]. Typically our mobile bases are instrumented with range sensors and/or cameras in addition to the wheel encoders and inertial measurement units (IMUs). The state estimation pipeline fuses these noisy measurements together into a consistent estimate. Indeed, one of the biggest lessons in mobile robots is the wheel odometry alone is almost always insufficient for estimation; real wheels slip and visual/range measurements provide an essential and independent set of measurements.

Many things have changed since [2] was written. Range-based sensors (lidar and depth cameras) have improved dramatically in range, accuracy, and framerate, making the methods dramatically more effective.

Example 7.2 (Simulating range sensors in Drake)

Perhaps the biggest change, though, has been the shift from a heavy dependence on range-based sensors / depth-cameras towards increasingly powerful estimation algorithm which use only RGB cameras. This is due to advances in visual-inertial odometry[4], monocular depth estimation[5], and dense 3D reconstruction / structure from motion[6, 7], to name a few.

7.3 What's different about motion planning?

Let's start with an example of perhaps the simplest (given our toolbox so far) to build ourselves a mobile manipulator.

Example 7.3 (iiwa on a prismatic base)

Let's take the iiwa model and add a few degrees of freedom to the base. Rather than welding link 0 to the world, let's insert three prismatic joints (connected by invisible, massless links) corresponding to motion in x, y, and z. I'll allow x and y to be unbounded, but will set modest joint limits on z. The first joint of the iiwa already rotates around the base z axis; I'll just remove the joint limits so that it can rotate continuously. Simple changes, and now I have the essence of a mobile robot.

Open in Colab

Now let's ask: how do I have to change my motion planning tools to work with this version of the iiwa? The answer is: not much! Our motion planning tools were all quite general, and work with basically any kinematic robot configuration. The mobile base that we've added here just adds a few more joints to that kinematic tree, but otherwise fits directly into the framework.

One possible consideration is that we have increased the number of degrees of freedom (here from 7 to 10). Trajectory optimization algorithms, by virtue(?) of being local to a trajectory, scale easily to high-dimensional problems, so this is no problem. Sampling-based planners struggle more with dimension, but RRTs should be fine in 10 dimensions. PRMs can definitely work in 10 dimensions, too, but probably require an efficient implementation and are starting to reach their limits.

Another consideration which may be new for the mobile base is the existence of a continuous joint (rotations around the z axis do not have joint limits). These can require some care in the planning stack. In a sampling-based planner, typically the

distance metric and extend operations are adapted to consider edges that can wrap around 2π. Nonconvex trajectory optimization doesn't require algorithmic modifications, per se, but does potentially suffer from local minima. For trajectory optimization with Graphs of Convex Sets, one simply needs to take care that the IRIS regions operate in a local coordinate system and don't have a domain wider than π in any wrapping joint[8]. Failure to address this properly can result in slightly absurd plans which take the long way around; here's an example of kinematic trajectory optimization finding a suboptimal solution to navigating between the shelves, from [8].

What happens if instead of implementing the mobile manipulator using prismatic joints, we're going to use wheels or legs? As we'll see, some wheeled robots (and most legged robots) can provide our prismatic joints abstraction directly, but others add constraints that we must consider during motion planning.

7.3.1 Wheeled robots

Even though real wheels do slip, we typically do motion planning with the no-slip assumption, then use feedback control to follow the planned path [9, Ch. 49].

Holonomic drives

Example 7.4 (Mecanum drive kinematics)

Example 7.5 (Simulating mecanum wheels in Drake)

Nonholonomic drives

Differential drive, Dubins car, Reeds-Shepp.

Example 7.6 (Differential drive kinematics)

7.3.2 Legged robots

If your job is to do the planning and control for the legs of a robot, then there is a rich literature for you to explore. But if you are using a legged robot like Spot to perform mobile manipulation, and you're using the API provided by Boston Dynamics, then (fortunately or unfortunately) you don't get to dive into the low-level control of the legs. In fact, Spot provides an API that to first order treats the legged platform as a holonomic base. (There are secondary considerations, like the fact that the robots center of mass is limited by dynamic constraints, and may deviate from your requested trajectory in order to maintain balance).

Example 7.7 (Mobile manipulation with Spot)

Of course, things get much more interesting when you are performing mobile manipulation on rough or intermitted terrain; this is where the legged platform really

shines!

7.4 What's different about simulation?

Mobile manipulation encourages us to expand the scope of our simulation, in addition to our autonomy algorithms. Rather than simulating a few objects on a table top, now we start asking about simulating the robot in a more expansive environment.

When I've used the term "simulation" so far in these notes, I've been focusing mostly on the enabling technologies, like the physics engine and the rendering engine. In robotics today, there are only a handful of mainstream simulators which people are using, such as NVidia Isaac/Omniverse, MuJoCo, and Drake. (PyBullet was very popular but is no longer actively maintained.) But there is another crop of tools which call themselves "simulators" that build upon these engines (and/or more traditional engines from computer games like Unity or Unreal) and offer higher-level abstractions. I think of these as content aggregators, and feel that they serve a very important role.

Example 7.8 (Sapien)

Example 7.9 (Behavior 1K)

Example 7.10 (Habitat 3.0)

For my part, I hope that the problems specifications and assets from these tools can be imported easily enough into Drake that we can use not only our simulation engine, but also our perception, planning, and control tools to solve them. We are gradually building out or support for loading from other formats, or translating from other formats in order to facilitate this.

7.5 Navigation

Most of the ideas we've covered in this chapter so far can be considered relatively modest extensions to our existing manipulation toolbox. But there are some important problems associated with navigating a mobile robot that really don't come up in table-top manipulation.

7.5.1 Mapping (in addition to localization)

7.5.2 Identifying traversable terrain

7.6 Exercises

Exercise 7.1 (Setting Up a Simulation)

For this exercise, you will load objects into a simulation to set up a useful manipulation scene. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Analyze the collision geometry of pre-defined SDF files.

b. Create your own simulation.

Exercise 7.2 (Mobile Base Inverse Kinematics)

For this exercise, you will implement an inverse kinematics optimization for a manipulator with both a fixed and mobile base. By removing position constraints on the base of our robot, we can solve the optimization much more easily. You will work exclusively in this notebook.

References

1. Armin Hornung and Kai M. Wurm and Maren Bennewitz and Cyrill Stachniss and Wolfram Burgard, "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees", *Autonomous Robots*, 2013.

2. S. Thrun and W. Burgard and D. Fox, "Probabilistic Robotics", MIT Press , 2005.

3. M. Kaess and A. Ranganathan and F. Dellaert, "iSAM: Incremental Smoothing and Mapping", *IEEE Transactions on Robotics*, vol. 24, no. 6, pp. 1365-1378, 2008.

4. Davide Scaramuzza and Friedrich Fraundorfer, "Visual odometry [tutorial]", *Robotics & Automation Magazine, IEEE*, vol. 18, no. 4, pp. 80--92, 2011.

5. Yue Ming and Xuyang Meng and Chunxiao Fan and Hui Yu, "Deep learning for monocular depth estimation: A review", *Neurocomputing*, vol. 438, pp. 14--33, 2021.

6. Ben Mildenhall and Pratul P Srinivasan and Matthew Tancik and Jonathan T Barron and Ravi Ramamoorthi and Ren Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis", *Communications of the ACM*, vol. 65, no. 1, pp. 99--106, 2021.

7. Bernhard Kerbl and Georgios Kopanas and Thomas Leimkuhler and George Drettakis, "3d gaussian splatting for real-time radiance field rendering", *ACM Transactions on Graphics (ToG)*, vol. 42, no. 4, pp. 1--14, 2023.

8. Thomas Cohn and Mark Petersen and Max Simchowitz and Russ Tedrake, "Non-Euclidean Motion Planning with Graphs of Geodesically-Convex Sets", *Robotics: Science and Systems* , 2023. [link]

9. Bruno Siciliano and Oussama Khatib, "Springer handbook of robotics", Springer , 2016.

Previous Chapter Table of contents Next Chapter

Accessibility

© Russ Tedrake, 2026